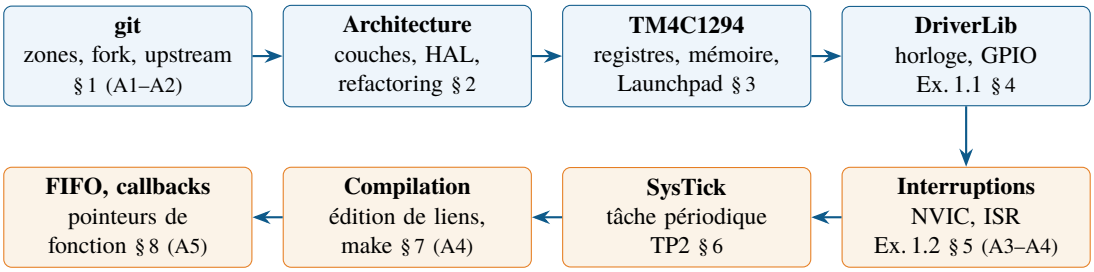
Systèmes embarqués 1 — Guide d'apprentissage TE1

git · architecture logicielle · TM4C1294 / Cortex-M4 · DriverLib · interruptions · SysTick

TE1 : mardi 27.10.2026, 13 :00–15 :45, D00.04 — compte pour 2/5 de la note (Note(CC) = (2 TE1 + 2 TE2 + Ra)/5). Matière : séances A1 à A5 (agenda officiel).

Sources : diapos SYE1-00/02/04 (R. Scherwey, D. Bullot), exercices 1.1 et 1.2 et leurs diagrammes, TP1, doc TivaWare (spmu298a), Yiu ch. 7, cours ALGO (pointeurs). Code C compilé (gcc strict) et exécuté sur un banc de test : `scripts/guide_te1_calculs.py` (33 contrôles C + 36 numériques).

Carte du TE1



Ce qui n'est pas dans l'arbre

Les supports des séances **A4** (compilation, édition de liens, FIFO) et **A5** (FIFO, pointeurs de fonction, callback) n'ont pas été déposés (ni OneDrive, ni Teams/OneNote miroirés). Les § 7–8 sont rédigés d'après la doc TI, le TP1 et le cours ALGO du même module : **compare-les avec tes notes de cours** et dépose les diapos si tu les as. Le dossier `TE` du OneDrive est vide : pas d'ancien TE disponible.

L'idée directrice en une phrase

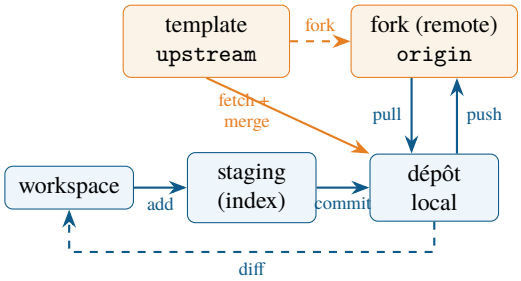
Un programme embarqué est un **empilement de couches** (application → HAL → DriverLib → matériel) qui tourne dans une **boucle infinie** ; les **interruptions** (boutons, SysTick) la « réveillent » par des ISR *courtes* qui ne font que lever un drapeau ou remplir une FIFO, et c'est `main` qui fait le travail.

1. git — gestion de versions (A1–A2, TP1)

Pourquoi : stocker un ensemble de fichiers, garder la **chronologie** des modifications, **retrouver** une version, et travailler **à plusieurs** (les modifications locales sont conservées et fusionnées).

Release : quand le projet est stable on pose un **tag** (RC1.0) ; on débogue la release (RC1.1, RC1.2) puis on reporte ces corrections dans la branche principale.

Organisation du cours : le prof publie un dépôt *template* sur la forge GitLab HEIA ; chacun en fait un **fork** (`Prénom.Nom/e2p_sye1_templates`), le **clone**, déclare le template comme remote **upstream**, et invite le prof comme *Reporter* (R. Scherwey pour E2-p2/E2-v).



Commande	Effet
`git config --global user.name "prénom.nom"`	identité utilisée dans chaque commit (idem `user.email`)
`git clone <url>`	copie locale du fork (dépôt + workspace)
`git status`	vert = indexé (sera commité) ; rouge = modifié non indexé ou non suivi
`git add f` / `git commit -m "msg"`	indexer, puis enregistrer un instantané *local* avec un message clair
`git push` / `git pull`	envoyer / rapatrier (fetch + merge) avec le remote `origin`
`git remote add upstream <url>`	déclarer le template du prof
`git fetch upstream` ; `git merge upstream/main`	récupérer puis fusionner les nouveaux exercices/TP du prof

TP1 : les fichiers `Debug__GNU/makefile` modifiés par CCS sont « rouges » ⇒ *discard* dans SmartGit ; les « verts » sont commités puis poussés.

2. Architecture logicielle (A2)

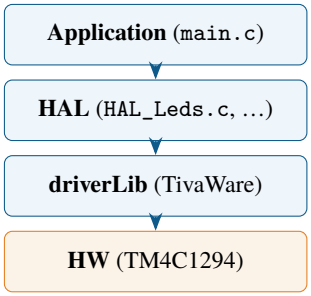
Définition : l'architecture ne décrit pas *ce que* fait le programme mais *comment* il est conçu pour répondre aux spécifications : **composants**, leurs **propriétés externes** et leurs **relations**.

Motivation : sans architecture, chaque changement ajoute des erreurs par **effets de bord** et le taux de défaillance du logiciel remonte ; avec architecture, il reste bas.

Types : appels-retours (décomposition fonctionnelle), **en couches**, centrée sur les données, en flot de données… Couches : 2 (client/serveur), 3 (présentation / métier / accès aux données, cf. MVC), 5 (+ application / service et objets de données).

Règle du cours : choisir la structure *avant* de coder (principalement en couches) ; une couche n'appelle que la couche du dessous. **CMSIS** : la couche d'abstraction standard d'ARM pour les Cortex-M.

Refactoring (réingénierie) : transformer et améliorer un logiciel existant — léger quand on reprend un développement, profond quand on le refond, « extrêmement souvent ». Procédure : **reverse engineering** (lire et analyser le code) → **forward engineering** (redéfinir selon la vue fonctionnelle) → **restructuring** (recoder) → **redocumentation**. Autre modèle : analyse des risques, rétro-ingénierie, reconstruction.



architecture globale du cours :
on n'appelle que la couche inférieure

3. Le microcontrôleur TM4C1294NCPDT et le Cortex-M4 (A2–A3)

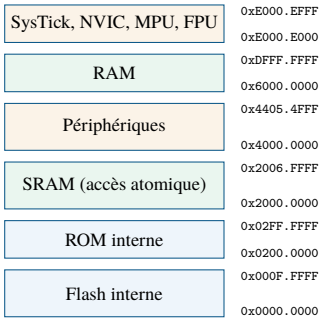
Plateforme EK-TM4C1294XL : Cortex-M4 **32 bits**, jusqu'à **120 MHz**.
Mémoires : 1024 ko flash, 256 ko SRAM, 6 ko EEPROM. **Communication** : 8 UART, 4 interfaces série synchrones, 10 I2C, 2 CAN, Ethernet, USB. **Système** : µDMA, 8 timers, 2 watchdogs, 90 GPIO, 1 module PWM (8 sorties), 2 ADC 12 bits à 2 Msps, 3 comparateurs analogiques.

Cœur Cortex-M4 : RISC, architecture **Harvard**, division matérielle, **SysTick** (timer système 24 bits), **NVIC** (jusqu'à 106 sources, 8 niveaux de priorité 0–7), **FPU** (simple précision = `float`, multiply-accumulate, +, −, ×, ÷, $\sqrt{}$, arrondis IEEE 754 ; pas de reste ni de conversions), **MPU**. Plus de 200 instructions (arithmétique, FPU, logique, sauts, load/store, sleep…).

Registres (32 bits) : **R0–R12** généraux ; **R13 = SP** (pointeur de pile : dernière donnée empilée) ; **R14 = LR** (adresse de retour après une fonction ou une interruption) ; **R15 = PC** (prochaine instruction). **PSR** : fanions de l'ALU, état du processeur et des interruptions ; **PRIMASK, FAULTMASK, BASEPRI** : masquage des exceptions ; **CONTROL** : contrôle du processeur.

Launchpad (schéma + guide utilisateur) : **D1 = PN1**, **D2 = PN0**, D3 = PF4, D4 = PF0 ; **USR_SW1 = PJ0**, **USR_SW2 = PJ1**, **actifs bas** (appui ⇒ niveau 0) ⇒ pull-up interne obligatoire. Deux emplacements BoosterPack.

Carte mémoire (diapo 10)



bus d'adresses 32 bits : 4 Gio adressables

4. DriverLib : horloge et GPIO en polling (Ex. 1.1)

Séquence d'initialisation (diagramme de l'exercice) : `main` → `hal_init()` → `clockInit()` [`SysCtlClockFreqSet`] → `buttonInit()` [`SysCtlPeripheralEnable`, `GPIOPinTypeGPIOInput`, `GPIOPadConfigSet`] → `ledInit()` [idem avec `GPIOPinTypeGPIOOutput`]. Boucle : `buttonState()` [`GPIOPinRead`] puis `ledWrite()` [`GPIOPinWrite`], « and same for SW2 ».

```
uint32_t g_sysClockHz;   /* fréquence réellement obtenue (valeur de retour) */

void clockInit(void)
{
    /* PLL à 480 MHz sur l'oscillateur interne 16 MHz, puis division -> 120 MHz */
    g_sysClockHz = SysCtlClockFreqSet(SYSCTL_OSC_INT | SYSCTL_USE_PLL |
                                      SYSCTL_CFG_VCO_480, SYS_CLOCK_HZ);
}

void HAL_Button_Init(void)
{
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOJ);           /* horloge du port J */
    while (!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOJ)) { }
    GPIOPinTypeGPIOInput(GPIO_PORTJ_BASE, GPIO_PIN_0 | GPIO_PIN_1);
    GPIOPadConfigSet(GPIO_PORTJ_BASE, GPIO_PIN_0 | GPIO_PIN_1,
                     GPIO_STRENGTH_2MA, GPIO_PIN_TYPE_STD_WPU);  /* pull-up */
}

bool buttonState(Button_t button)
{
    uint8_t pin = (button == eBUTTON_USR_SW1) ? GPIO_PIN_0 : GPIO_PIN_1;
    return GPIOPinRead(GPIO_PORTJ_BASE, pin) == 0;          /* actif bas */
}

void HAL_Led_Init(void)
{
    SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION);
```

```
    while (!SysCtlPeripheralReady(SYSCTL_PERIPH_GPION)) { }
    GPIOPinTypeGPIOOutput(GPIO_PORTN_BASE, GPIO_PIN_0 | GPIO_PIN_1);
    GPIOPadConfigSet(GPIO_PORTN_BASE, GPIO_PIN_0 | GPIO_PIN_1,
                     GPIO_STRENGTH_6MA, GPIO_PIN_TYPE_STD);     /* 6 mA */
}

void HAL_Led_Write(ledType_t led, bool state)
{
    uint8_t pin = (led == eLED1) ? GPIO_PIN_1 : GPIO_PIN_0;  /* D1 = PN1, D2 = PN0 */
    GPIOPinWrite(GPIO_PORTN_BASE, pin, state ? pin : 0);     /* valeur bit à bit */
}

void init(void)
{
    clockInit();
    HAL_Button_Init();
    HAL_Led_Init();
}
```

```
int main(void)
{
    init();                         /* clockInit, HAL_Button_Init,
     HAL_Led_Init */
    while (1)
    {
        /* polling pur : on recopie l'état des boutons sur les LED
       */
        HAL_Led_Write(eLED1, buttonState(eBUTTON_USR_SW1));
        HAL_Led_Write(eLED2, buttonState(eBUTTON_USR_SW2));
    }
}
```

À savoir expliquer :

- **masques « bit-packed »** : `GPIO_PIN_0` = 0x01, `GPIO_PIN_1` = 0x02 ; on combine par `|` ;
- `GPIOPinWrite(port, pins, val)` n'écrit *que* les broches de `pins` : allumer PN1 = `GPIOPinWrite(N, GPIO_PIN_1, GPIO_PIN_1)`, l'éteindre = `(…, GPIO_PIN_1, 0)` ; écrire 1 n'allume que PN0 !
- `GPIOPinRead` renvoie le *masque* (0x02 pour PJ1), pas 1 : comparer à 0 ;
- actif bas : `== 0` signifie « appuyé » ; **pull-up** `GPIO_PIN_TYPE_STD_WPU` ;
- pour une entrée, seul le type (pull-up/down) de `GPIOPadConfigSet` compte, pas l'intensité ;
- `SysCtlClockFreqSet` *renvoie* la fréquence obtenue (à garder pour SysTick, UART…) ;
- sans `SysCtlPeripheralEnable`, l'accès au port provoque une **faute** (exception).

5. Interruptions et exceptions (A3–A4, Ex. 1.2)

Polling : le CPU teste sans cesse l'entrée (simple, mais il ne fait que ça et peut rater un événement bref). **Interruption** : le périphérique *demande* le service ; le CPU suspend la tâche, exécute l'**ISR** (Interrupt Service Routine), puis reprend.

Vocabulaire ARM : une interruption est un type d'**exception**. Numéros 1–15 : exceptions système ; 16 et plus : IRQ des périphériques (IRQ n = exception 16 + n).

N°	Exception	Priorité
1	Reset	−3 (la plus haute)
2	NMI	−2
3	HardFault	−1
4–6	MemManage, BusFault, UsageFault	programmable
11 / 14	SVC / PendSV	programmable
15	**SysTick**	programmable
16+	IRQ (GPIO, UART, timers…)	programmable

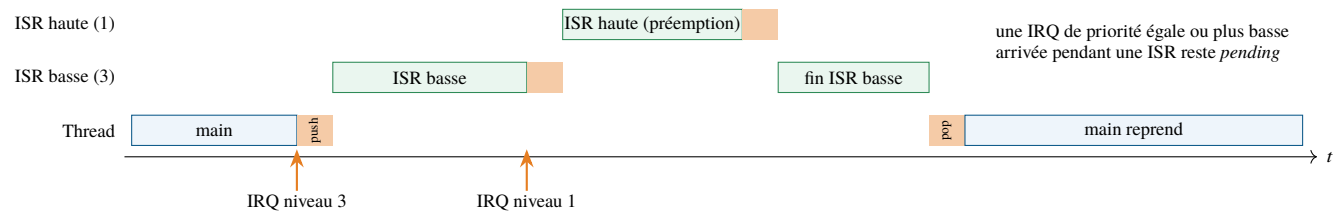
Table des vecteurs (adresse 0) : mot 0 = SP initial, mot 1 = vecteur Reset, … mot k = adresse du handler de l'exception k (SysTick à l'offset 0x3C, IRQ0 à 0x40).

Priorités : **plus petit** = **plus prioritaire**. Le TM4C n'implémente que **3 bits** (8 niveaux), les bits de *poids fort* de l'octet : `IntPrioritySet(INT_GPIOJ, 2 << 5)` écrit 0x40 (niveau 2) ; 0xE0 = niveau 7 (le plus bas).

Une exception est acceptée si : le processeur tourne ; elle est **autorisée** ; sa priorité est **plus haute** que le niveau courant ; elle n'est pas **masquée** (PRIMASK, BASEPRI).

États : *pending* (en attente) et/ou *active* (en cours). Le pending est mémorisé même si la requête retombe ; plusieurs impulsions avant le service = **une** seule ISR ; un pending peut exister alors que l'IRQ est désactivée (l'effacer avant d'activer si nécessaire).

Entrée : **empilement** automatique de R0–R3, R12, LR, PC, xPSR (8 mots), lecture du vecteur, mode *Handler* (pile MSP, privilégié), LR = **EXC_RETURN**. **Sortie** : retour normal de fonction C ⇒ EXC_RETURN dans PC ⇒ **dépilement**. C'est pourquoi une ISR s'écrit comme une fonction C `void f(void)`.



Ex. 1.2 — boutons sur **les deux flancs** : l'ISR enregistre l'événement, `main` met à jour les LED (diagrammes de séquence de l'exercice : `GPIOIntRegister`, `GPIOIntTypeSet`, `GPIOIntEnable` ; ISR : `GPIOIntStatus`, `GPIOIntClear`, `buttonState`).

```
static volatile ButtonEvent_t usrSw1Event = eBUTTON_NO_EVENT;  /* partagé ISR/main */
static volatile ButtonEvent_t usrSw2Event = eBUTTON_NO_EVENT;
```

```
static void button_ISR(void)
{
    uint32_t status = GPIOIntStatus(GPIO_PORTJ_BASE, true);   /* qui a déclenché ? */
    if (status & GPIO_INT_PIN_0) {
        GPIOIntClear(GPIO_PORTJ_BASE, GPIO_INT_PIN_0);        /* acquitter TÔT */
        usrSw1Event = buttonState(eBUTTON_USR_SW1) ? eBUTTON_EVENT_PRESSED
                                                   : eBUTTON_EVENT_RELEASED;
    }
    if (status & GPIO_INT_PIN_1) {
        GPIOIntClear(GPIO_PORTJ_BASE, GPIO_INT_PIN_1);
        usrSw2Event = buttonState(eBUTTON_USR_SW2) ? eBUTTON_EVENT_PRESSED
                                                   : eBUTTON_EVENT_RELEASED;
    }
}

void HAL_Button_InitIrq(void)
{
    HAL_Button_Init();                                          /* entrées + pull-up */
    GPIOIntRegister(GPIO_PORTJ_BASE, button_ISR);               /* + active l'IRQ du port J */
    GPIOIntTypeSet(GPIO_PORTJ_BASE, GPIO_PIN_0 | GPIO_PIN_1, GPIO_BOTH_EDGES);
    GPIOIntEnable(GPIO_PORTJ_BASE, GPIO_INT_PIN_0 | GPIO_INT_PIN_1);
    IntMasterEnable();                                          /* PRIMASK = 0 */
}

ButtonEvent_t buttonEvent(Button_t button)   /* lit ET consomme l'événement */
{
    ButtonEvent_t ev;
    IntMasterDisable();                      /* section critique : lecture + remise à zéro */
    if (button == eBUTTON_USR_SW1) { ev = usrSw1Event; usrSw1Event = eBUTTON_NO_EVENT; }
    else                           { ev = usrSw2Event; usrSw2Event = eBUTTON_NO_EVENT; }
    IntMasterEnable();
    return ev;
}

void app_ex12_step(void)                     /* corps du while(1) de main */
{
    ButtonEvent_t ev = buttonEvent(eBUTTON_USR_SW1);
    if (ev == eBUTTON_EVENT_PRESSED)  { HAL_Led_Write(eLED1, true); }
    if (ev == eBUTTON_EVENT_RELEASED) { HAL_Led_Write(eLED1, false); }
    ev = buttonEvent(eBUTTON_USR_SW2);
    if (ev == eBUTTON_EVENT_PRESSED)  { HAL_Led_Write(eLED2, true); }
    if (ev == eBUTTON_EVENT_RELEASED) { HAL_Led_Write(eLED2, false); }
}
```

Les quatre pièges de l'ISR

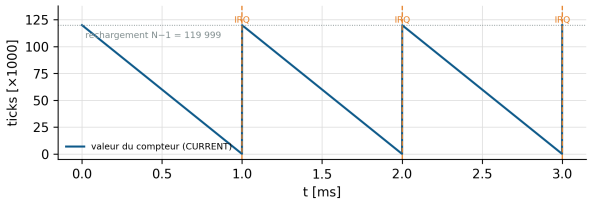
(1) `volatile` sur toute variable partagée ISR/`main` : sinon le compilateur peut la garder dans un registre et `main` ne voit jamais le changement. **(2) Acquitter tôt** (`GPIOIntClear` en début d'ISR) : à cause du tampon d'écriture du Cortex-M, un acquittement en dernière instruction peut faire ré-entrer l'ISR. **(3) Section critique** : « lire puis remettre à `NO_EVENT` » n'est pas atomique ; une IRQ entre les deux ferait perdre un événement. **(4) Rebonds** mécaniques : un appui génère plusieurs flancs ⇒ plusieurs événements (anti-rebond par temporisation SysTick).

6. SysTick : le temps réel simple (A3–A5, TP2 du 15.10)

Compteur **24 bits décroissant**, intégré au cœur, qui compte même pendant les interruptions ; à chaque passage par 0 il se recharge à $N - 1$ et lève l'exception 15. Usages : tick d'un RTOS, alarme rapide, **mesure de temps d'exécution**.

$$N = f_{clk} \cdot T \quad (T \text{ : période voulue}), \qquad 1 \le N \le 2^{24} = 16\,777\,216$$

À 120 MHz : 1 ms → $N = 120\,000$; 10 ms → 1 200 000 ; période maximale $2^{24}/120\,\text{MHz} = 139{,}8$ ms (1,05 s à 16 MHz). Au-delà : compter les ticks dans l'ISR.



Mesure : lire `SysTickValueGet()` avant/après ; comme le compteur *décroît*, écoulé = début − fin (modulo N) ; 3000 ticks = 25 μs.

```
static volatile bool systickFlag = false;

__interrupt void sysTickIntHandler(void)
{
    systickFlag = true;                 /* court : le travail se fait dans main */
}

void HAL_TM4C1294_SystickInit(uint32_t us)
{
    /* nombre de ticks d'horloge par période ; 1 <= N <= 2^24 = 16 777 216 */
    SysTickPeriodSet((g_sysClockHz / 1000000u) * us);
    SysTickIntRegister(sysTickIntHandler);   /* le nom passé doit être le handler */
    SysTickIntEnable();
    SysTickEnable();
}

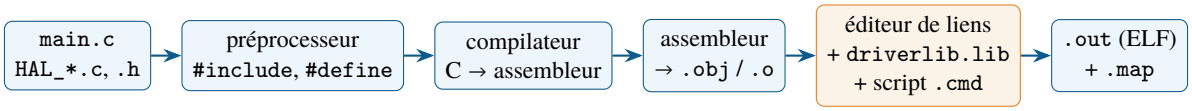
bool systickElapsed(void)               /* vrai une fois par période */
```

```
{
    bool fired = false;
    IntMasterDisable();
    if (systickFlag) { systickFlag = false; fired = true; }
    IntMasterEnable();
    return fired;
}

/* compteur DÉCROISSANT : écoulé = (début - fin) modulo la période */
uint32_t ticksElapsed(uint32_t start, uint32_t end, uint32_t period)
{
    return (start >= end) ? (start - end) : (start + period - end);
}
```

Diapo 13 : `SysTickIntRegister(sysTickIntHandler)` — « le nom de la fonction doit correspondre » ; mot-clé `__interrupt` du compilateur TI. Pas besoin d'acquitter l'interruption SysTick (doc TI). Le handler ne fait que lever un drapeau : c'est l'architecture « super-boucle + tick » des TP.

7. Compilation, édition de liens et make (A4)



Compilation : un fichier `.c` à la fois ; un appel à une fonction seulement *déclarée* (prototype du `.h`) est laissé en suspens. **Édition de liens** : résout ces symboles entre les `.obj` et les bibliothèques, puis place chaque section en mémoire selon le script de liens.

Sections : `.text` (code) et constantes en **flash** ; `.data` (variables initialisées, copiées de la flash vers la RAM au démarrage), `.bss` (mises à zéro), pile et tas en **RAM**. Le fichier `.map` donne adresse et taille de chaque symbole (TP1 : `grep -n -A 1 'main' Debug/TP1-Tools.map`).

Erreurs : *compilation* (syntaxe, type, prototype manquant) vs *édition de liens* (« undefined symbol » : fonction déclarée jamais définie / bibliothèque absente ; « multiple definition » : variable *définie* dans un `.h`).

Mots-clés : `static` (portée limitée au fichier, ou variable locale persistante), `extern` (déclaration d'une globale définie ailleurs), `volatile` (§ 5), `const` (en flash), `inline`. Garde d'inclusion `#ifndef X_H / #define X_H / #endif`.

make : reconstruit une cible quand une dépendance est plus récente (`make clean`, `make` ; `touch ../src/main.c` force la recompilation). Options : niveau d'optimisation (taille vs vitesse), symboles de débogage.

8. FIFO, pointeurs de fonction et callbacks (A5)

```
#define FIFO_SIZE 8u                        /* puissance de 2 : masque
      au lieu de modulo */

typedef struct {
    uint8_t data[FIFO_SIZE];
    volatile uint32_t head;                 /* prochaine case à écrire
      (producteur) */
    volatile uint32_t tail;                 /* prochaine case à lire (
      consommateur) */
} Fifo_t;
```

```
void fifoInit(Fifo_t *f) { f->head = 0; f->tail = 0; }

bool fifoIsEmpty(const Fifo_t *f) { return f->head == f->tail; }

bool fifoIsFull(const Fifo_t *f)
{
    return ((f->head + 1u) & (FIFO_SIZE - 1u)) == f->tail;   /*
      capacité = FIFO_SIZE - 1 */
}

bool fifoPut(Fifo_t *f, uint8_t value)      /* appelé par l'ISR (
      producteur) */
{
    if (fifoIsFull(f)) { return false; }      /* donnée perdue : à
      signaler */
    f->data[f->head] = value;                 /* écrire D'ABORD la
      donnée */
    f->head = (f->head + 1u) & (FIFO_SIZE - 1u);   /* …PUIS publier
      l'indice */
    return true;
}

bool fifoGet(Fifo_t *f, uint8_t *value)     /* appelé par main (
      consommateur) */
{
    if (fifoIsEmpty(f)) { return false; }
    *value = f->data[f->tail];
    f->tail = (f->tail + 1u) & (FIFO_SIZE - 1u);
    return true;
}
```

Pourquoi une FIFO : l'ISR (producteur) dépose des données plus vite ou à d'autres moments que `main` (consommateur) ne les traite — typiquement les octets d'un UART. Premier entré, premier sorti.

Tampon circulaire : `head` (écriture) et `tail` (lecture) reviennent à 0 ; vide si `head == tail`, plein si `head + 1 == tail` (modulo) : **une case reste vide**, capacité $N - 1$ (7 pour 8 cases). Taille puissance de 2 ⇒ masque `& (N-1)` au lieu de `%`.

Un producteur, un consommateur : chacun ne modifie que *son* indice ; écrire la donnée *avant* de publier l'indice ⇒ pas besoin de bloquer les interruptions.

Pointeur de fonction : variable contenant l'adresse d'une fonction (cours ALGO) : `typedef int16_t (*Function_t)(int16_t, int16_t);` ; affectation `f = maFonction;` ; appel `f(10, 20)` ou `(*f)(10, 20)` ; tableau `Function_t ops[] = {add, sub};`.

Callback : la couche basse *reçoit* l'adresse d'une fonction de la couche haute et l'appelle plus tard — la HAL n'a pas besoin de connaître l'application (couches respectées). `SysTickIntRegister(handler)` et `GPIOIntRegister(port, isr)` sont exactement cela.

```c
/* typedef void (*ButtonCallback_t)(Button_t button, ButtonEvent_t event);  (hal.h) */
static ButtonCallback_t s_callback = NULL;          /* pointeur de fonction */

void HAL_Button_RegisterCallback(ButtonCallback_t callback)
{
    s_callback = callback;                          /* « enregistrer » = mémoriser l'adresse */
}

void HAL_Button_Notify(Button_t button, ButtonEvent_t event)
{
    if (s_callback != NULL) {                       /* toujours tester NULL */
        s_callback(button, event);                  /* appel indirect = (*s_callback)…() */
    }
}

/* Tableau de pointeurs de fonction (cours ALGO) */
typedef int16_t (*Function_t)(int16_t, int16_t);
static int16_t add(int16_t a, int16_t b) { return (int16_t)(a + b); }
static int16_t sub(int16_t a, int16_t b) { return (int16_t)(a - b); }
static const Function_t s_ops[] = { add, sub };

int16_t compute(uint8_t op, int16_t a, int16_t b) { return s_ops[op](a, b); }
```

Attention : un callback appelé depuis une ISR s'exécute *dans* l'ISR : il doit rester court.

9. Codage des nombres (objectif 4 du descriptif — à vérifier s'il est au TE1)

Non signé sur n bits : $0 \dots 2^n - 1$; **complément à deux** : $-2^{n-1} \dots 2^{n-1} - 1$ (`int8_t` : $-128 \dots 127$; `int16_t` : $-32\,768 \dots 32\,767$). Opposé : inverser les bits puis $+1$. 0xFF = 255 (non signé) = -1 (signé) ; 0x80 = -128 ; 0xF6 = -10.
Débordement : `uint8_t` 200 + 100 = 44 (modulo 256) ; `int8_t` 100 + 100 → -56.

Virgule fixe Q1.15 : valeur = entier$/2^{15}$; 0,5 → 0x4000, $-0{,}25$ → 0xE000.
Virgule flottante IEEE 754 simple (`float`) : 1 bit de signe, 8 d'exposant (biais 127), 23 de mantisse ; 1,0f = 0x3F800000, $-2{,}5$f = 0xC0200000. La FPU du M4 ne traite que la **simple** précision : écrire `2.5f` et `float` (un `double` serait émulé en logiciel, lentement).

10. Mémo des pièges

1. `SysCtlPeripheralEnable` avant tout accès au port.
2. D1 = PN1 et D2 = PN0 (ordre inversé !).
3. Boutons actifs bas : appuyé ⇔ lecture = 0 ; pull-up interne.
4. `GPIOPinWrite(…, GPIO_PIN_1, 1)` n'allume rien : écrire le masque.
5. `GPIOPinRead` rend un masque, pas un booléen.
6. Variable partagée avec une ISR : `volatile`.
7. Acquitter l'interruption *au début* de l'ISR.
8. ISR courte : pas d'attente active, pas de `printf`.
9. Priorité : 0 = la plus haute ; 3 bits de poids fort.
10. `GPIOIntRegister` active l'IRQ dans le NVIC, `GPIOIntEnable` la source de broche ; `IntMasterEnable` lève le masque global.
11. SysTick : 24 bits, décroissant, $N \le 2^{24}$.
12. « undefined symbol » = édition de liens, pas compilation.
13. FIFO : capacité $N - 1$.
14. git : `commit` est local, `push` publie.

11. Auto-test (réponses en bas de page)

1. Donner, dans l'ordre, les appels DriverLib pour allumer la LED D2.
2. Configurer SysTick pour 5 ms à 120 MHz. Peut-on obtenir 200 ms en une seule période ?
3. Pourquoi `volatile` ? Que se passe-t-il sans ?
4. Deux IRQ pendantes, niveaux 1 et 3 : laquelle est servie ? Et si la 3 arrive pendant l'ISR de la 1 ?
5. Que vaut 0xF6 en `int8_t` ? en `uint8_t` ?
6. Différence entre `git fetch upstream` et `git pull` ? entre `commit` et `push` ?
7. Citer les 4 couches de l'architecture du cours et y placer `buttonState()`.
8. Une FIFO de 16 cases : combien d'octets au maximum ? Pourquoi ?

Réponses. 1) `SysCtlPeripheralEnable(SYSCTL_PERIPH_GPION)` (+ `Ready`), `GPIOPinTypeGPIOOutput(GPIO_PORTN_BASE, GPIO_PIN_0)`, `GPIOPadConfigSet(…, GPIO_STRENGTH_6MA, GPIO_PIN_TYPE_STD)`, `GPIOPinWrite(GPIO_PORTN_BASE, GPIO_PIN_0, GPIO_PIN_0)`. 2) `SysTickPeriodSet(600000)` puis register/IntEnable/Enable ; non : max 139,8 ms ⇒ compter 40 ticks de 5 ms. 3) Le compilateur peut mettre la variable en registre et ne jamais relire la mémoire modifiée par l'ISR. 4) La 1 (plus petit = plus prioritaire) ; la 3 reste pending jusqu'à la fin de l'ISR 1. 5) -10 ; 246. 6) `fetch` rapatrie sans fusionner, `pull` = fetch + merge ; `commit` enregistre localement, `push` envoie au serveur. 7) Application, HAL, driverLib, HW ; `buttonState` est dans la HAL. 8) 15 : une case reste vide pour distinguer plein et vide.